

o MILESTONE 3: API Logic Integration – LegalEase

In this milestone, we implement the FastAPI backend, validate incoming document details, connect the API route to the Gemini document generator, and return the generated document as a JSON response.

Activity 3.1: Create the FastAPI Application

1. Initialize the Backend

Create a FastAPI app and include the routes module.

main.py

```
from fastapi import FastAPI
from routes import router

app = FastAPI(title="LegalEase API")
app.include_router(router)

@app.get("/")
def home():
    return {"message": "LegalEase API is running"}
```

Activity 3.2: Define and Validate the Request

1. Create the Request Schema

Use Pydantic to define the document fields expected in the JSON request.

schemas.py

```
from pydantic import BaseModel

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str
```

2. Request Data

The endpoint receives document_type, parties, terms, and dates from the frontend.

Activity 3.3: Implement the Generate Route

1. Create POST /generate

Call the Gemini generator with the validated request fields and return the result in a document key.

routes.py

```
from fastapi import APIRouter
from schemas import DocumentRequest

router = APIRouter()

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
```

```
)  
return {"document": response}
```

Activity 3.4: Connect the Gemini Generator

1. Generate the Document

Construct the prompt, send it to the configured Gemini model, and return the response text.

gemini_generator.py

```
def generate_document(self, document_type, parties, terms, dates):  
    prompt = (  
        f"Generate a comprehensive legal document titled '{document_type}'\  
    \n"  
        f"Involved parties: {parties}\\n"  
        f"Effective Date: {dates}\\n"  
        f"Terms and conditions: {terms}\\n"  
        "Ensure formal legal structure with multiple sections and legal cla  
uses."  
    )  
    response = self.model.generate_content(prompt)  
    return response.text
```

2. Return the API Response

The backend returns a JSON object: {"document": "generated document text"}.